

Implementation Plan With Timeline

Phase	Focus Area	Key Deliverables & Git Commit Goals	Target Duration
Phase 1	Setup & Data Layer	• Initialize FastAPI, SQLAlchemy 2.0 config, and SQLite engine with WAL mode (PRAGMA journal_mode=WAL ;) for high concurrency. • Define Employee model with fields: id, name, email, job_title, department, country, employment_type, base_salary, bonus, currency, salary_usd, status, is_deleted, created_at, and updated_at. • Configure composite and single-column indexes on country, department, job_title, employment_type, status, is_deleted, and salary_usd. • Commit: feat(db): configure sqlite wal mode and complete	1.0–1.5 Hours

		employee schema with composite indexing	
Phase 2	Batch Seeding Script	• Create scripts/seed.py using Faker with realistic multi-country compensation data and static base-currency exchange rates. • Implement high-performance bulk insertion using SQLAlchemy 2.0 session.execute(insert(Employee), [...]) in chunks to seed 10,000 records in <2s. • Commit: feat(seed): add high-performance 10k employee seed script with static fx mapping	1.0 Hour
Phase 3	Core Service, CRUD & CSV Export APIs	• Implement Pydantic validation schemas (positive numeric bounds, valid currency codes, required fields) and USD normalization helpers. • Build paginated	2.0 Hours

		list endpoint (GET /api/employees) supporting pagination, multi-attribute filtering (department, country, job_title, employment_type, status), search, and sorting (base_salary, bonus, salary_usd). • Build CRUD endpoints: POST /api/employees, GET /api/employees/:id, PUT /api/employees/:id, and soft-delete DELETE /api/employees/:id (sets is_deleted=True). • Implement GET /api/employees/export streaming endpoint to download filtered search results as a CSV file. • Commit: feat(api): implement employee crud, soft-delete, advanced filtering, and csv export	
--	--	---	--

Phase 4	Analytics & Aggregations	• Write optimized SQL aggregation queries (COUNT, AVG, SUM, MIN, MAX) grouped by department and country, filtering out soft-deleted records (is_deleted=False). • Implement median salary calculation using SQL window functions (PERCENT_RANK / NTILE) or sorted middle-element SQL query. • Expose /api/analytics/summary (including total annual payroll, average, median salary, and currency distribution), /api/analytics/by-department, and /api/analytics/by-country. • Commit: feat(analytics): add org salary aggregations, median calculations, and currency distribution apis	1.5 Hours
Phase 5	Testing & CI Readiness	• Write deterministic Pytest unit tests for	1.5 Hours

		static FX salary normalization, median calculations, and Pydantic validation rules. • Write integration tests using TestClient with an in-memory SQLite database (sqlite:///memory:) covering pagination boundaries, filtering, soft-deletion behavior, CSV export, and aggregate analytics. • Commit: test(backend): add unit and integration test suite covering crud, filters, exports, and analytics	
Phase 6	Performance & Documentation	• Benchmark and verify query latency across the 10,000 seeded dataset to guarantee <200ms response targets on pagination and analytics queries. • Finalize OpenAPI docs, CORS settings for Next.js/React frontend integration, and backend	0.5–1.0 Hour

		README.md artifacts. • Commit: docs(backend): add setup instructions, benchmark verification, seed guide, and api specs	
--	--	---	--

Execution Strategy & Best Practices

- **Keep Commits Atomic:** Follow the commit cadence outlined above to showcase clear, intentional architectural progress in Git history.
- **Ensure Concurrency Safety:** Explicitly enable WAL mode on SQLite connection initialization to ensure analytical aggregation queries do not lock out write CRUD requests.
- **Zero External Dependencies for FX:** Use a fixed conversion matrix module (constants/fx_rates.py) to satisfy standardized base-currency requirements deterministically without third-party network failure points.
- **Filter Out Soft Deletions Universally:** Centralize query filters in the repository/service layer so that all pagination, export, and KPI aggregations exclude records where `is_deleted == True`.

Project Directory Structure

```
None
backend/
├── app/
│   ├── __init__.py
│   ├── config.py # Environment variables & app settings
│   ├── database.py # SQLAlchemy engine & SQLite WAL config
│   ├── models.py # SQLAlchemy entities (with soft-delete & indexing)
│   ├── schemas.py # Pydantic request/response validators
│   ├── constants/
│   │   ├── __init__.py
│   │   └── fx_rates.py # Static currency conversion lookup matrix
│   ├── services/ # Business Logic Layer
│   │   ├── __init__.py
│   │   ├── employee_service.py # CRUD, soft-delete, filtering & CSV generation
│   │   └── analytics_service.py # Aggregations, median & currency distribution
│   ├── routers/ # API Route Handlers
│   │   ├── __init__.py
│   │   ├── employees.py # CRUD, list, and CSV export endpoints
│   │   └── analytics.py # Summary, department, & country analytics
│   └── main.py # FastAPI entrypoint & CORS middleware
└── scripts/
```

```
|   └─ seed.py # Optimized 10k employee batch seeder
└─ tests/
    ├── __init__.py
    ├── conftest.py # In-memory test db & client fixtures
    ├── test_employees.py # Unit/Integration tests for CRUD & CSV export
    └── test_analytics.py # Unit/Integration tests for KPIs & medians
└─ requirements.txt
└─ README.md
```